

BÁO CÁO ĐỒ ÁN

MÔN: THỰC HÀNH CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

HK3 - NĂM HỌC: 2025-2026

ĐỒ ÁN NHÓM 11:

HỆ THỐNG BÃI GIỮ XE THÔNG MINH VÀ DẪN ĐƯỜNG (SMART CAMPUS PARKING)

LỚP: 24DTV_DKD2

GVHD: ThS. Trần Tuấn Kiệt

STT	Họ và tên	MSSV
1	Lê Trần Khánh An	22207001
2	Nguyễn Huân	23207002
3	Nguyễn Tuấn Kiệt	23207008
4	Lê Minh Trọng	23207030
5	Lâm Trung Hiếu	23207054
6	Nguyễn Bá Thành	23207106
7	Mai Lý Khải Triều	23207136

MỤC LỤC

LỜI CẢM ƠN	3
DANH MỤC HÌNH.....	4
I. GIỚI THIỆU	5
II. CƠ SỞ LÝ THUYẾT	6
2.1. Danh sách liên kết đơn	6
2.1.1. Khái niệm.....	6
2.1.2. Mã giả	6
2.2. Ngăn xếp (Stack)	6
2.2.1. Khái niệm.....	6
2.2.2. Mã giả	6
2.3. Hàng đợi (Queue)	7
2.3.1. Khái niệm.....	7
2.3.2. Mã giả	7
2.4. Bảng băm (Hash Table).....	7
2.4.1. Khái niệm.....	7
2.4.2. Mã giả	8
2.5. Separate Chaining	8
2.5.1. Khái niệm.....	8
2.5.2. Mã giả	8
2.6. Ma trận kề (Adjacency Matrix)	9
2.6.1. Khái niệm.....	9
2.6.2. Mã giả	9
2.7. Hàng đợi ưu tiên (Priority Queue).....	9
2.7.1. Khái niệm.....	9
2.7.2. Mã giả	10
2.8. Heap (Min Heap)	10
2.8.1. Khái niệm.....	10
2.8.2. Mã giả	11
2.9. Quy hoạch động (Dynamic Programming)	11
2.10. Khoảng cách Levenshtein	11

2.10.1. Khái niệm	11
2.10.2. Mã giả	11
2.11. Thuật toán Bubble Sort.....	12
2.11.1. Khái niệm	12
2.11.2. Mã giả	12
2.12. Thuật toán Fuzzy Search	13
2.12.1. Khái niệm	13
2.12.2. Mã giả	13
2.13. Thuật toán Shortest Path Dijkstra	14
2.13.1. Khái niệm	14
2.13.2. Mã giả	14
2.14. Đồ thị (Graph)	16
2.14.1. Khái niệm	16
2.14.2. Mã giả	16
III. MÔ HÌNH HOẠT ĐỘNG	17
IV. KẾT QUẢ.....	21
IV. PHÂN TÍCH - ĐÁNH GIÁ	24
V. TÀI LIỆU THAM KHẢO	27

LỜI CẢM ƠN

Lời đầu tiên, nhóm chúng em xin gửi lời cảm ơn chân thành đến thầy/cô Khoa Điện tử - Viễn thông đã tạo điều kiện thuận lợi và môi trường học tập tốt nhất để chúng em tiếp cận với học phần Thực hành Cấu trúc dữ liệu và giải thuật.

Đặc biệt, chúng em xin bày tỏ lòng biết ơn sâu sắc đến thầy Trần Tuấn Kiệt. Trong suốt quá trình học tập, thầy không chỉ truyền đạt nền tảng kiến thức lý thuyết vững chắc mà còn tận tình hướng dẫn chúng em phương pháp tư duy giải quyết vấn đề. Sự chỉ dẫn của thầy đã giúp nhóm hoàn thành tốt các yêu cầu trong dự án này.

Dù cố gắng hết sức tuy nhiên do vốn kiến thức tích lũy còn hạn chế nên bản báo cáo đồ án của nhóm em sẽ không thể tránh khỏi những sai sót. Chúng em mong được thầy xem xét và đóng góp ý kiến để bài báo cáo của nhóm được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

TPHCM, ngày 22/07/2026

DANH MỤC HÌNH

Hình 1. Lưu đồ giải thuật các cấu trúc dữ liệu cơ sở (HashTable, Queue FIFO, Stack LIFO, Min-Heap).	17
Hình 2. Lưu đồ giải thuật hàm InsertHash	17
Hình 3. Lưu đồ giải thuật heapifyUp và heapifyDown.....	17
Hình 4. Lưu đồ giải thuật findNearestSlot	17
Hình 5. Lưu đồ giải thuật quy trình xe vào bãi (processCheckIn)	18
Hình 6. Lưu đồ giải thuật findAndEvictStudent	18
Hình 7. Lưu đồ giải thuật fuzzySearchLicensePlate.....	19
Hình 8. Lưu đồ giải thuật processCheckOut.....	20
Hình 9. Lưu đồ giải thuật normalizeString.....	20
Hình 10. Lưu đồ giải thuật getLevenshteinDistance	21
Hình 11. Lưu đồ giải thuật main()	21
Hình 12. Giao diện tổng thể.....	21
Hình 13. Kết quả khi bãi đỗ sinh viên được lấp đầy.....	21
Hình 14. Kết quả khi bãi đỗ Giảng viên được lấp đầy.....	22
Hình 15. Kết quả khi bãi xe được lấp đầy	22
Hình 16. Khi bấm vào 1 ô bất kì thì sẽ có thông tin ô đỗ bao gồm mã thẻ, biển số xe, loại xe, khu đỗ và thời gian vào	22
Hình 17. Khi bãi xe sinh viên đầy thì xe sinh viên đậu vào bãi của GV.....	22
Hình 18. Trường hợp bãi xe Giảng viên đã đầy thì phải di dời xe SV đậu nhờ để cho Giảng viên khác vào	23
Hình 19. Tìm kiếm thông qua biển số xe hoặc mã số thẻ	23
Hình 20. Thông báo trùng biển số	23
Hình 21. Thông báo trùng mã thẻ.....	23
Hình 22. Khi bãi xe đã đầy thì xe vào sau sẽ được đỗ tạm bên ngoài bãi xe	24
Hình 23. Khi có một xe trong bãi ra khỏi bãi đỗ thì xe trong hàng đợi sẽ được đưa vào bãi xe theo FIFO	24
Hình 24. Xe mang biển số 59XB04160 vào ô 101.....	24

I. GIỚI THIỆU

Quản lý bãi đỗ xe quy mô lớn trong môi trường trường học yêu cầu hệ thống vận hành chính xác, có khả năng đồng bộ dữ liệu nhanh chóng và tối ưu hóa điều phối giao thông nội khu. Hiện nay, các phương pháp quản lý thủ công trong ghi nhận xe ra/vào, kiểm soát sức chứa và tìm kiếm phương tiện thường tiềm ẩn rủi ro sai sót dữ liệu, dẫn đến tình trạng ùn tắc cục bộ vào giờ cao điểm. Bên cạnh đó, việc thiếu kiến trúc phần mềm đồng nhất gây ra nhiều hạn chế trong việc tự động hóa dẫn đường, xử lý dữ liệu nhiều từ hệ thống camera nhận diện biển số và phân bổ không gian đỗ xe giữa các nhóm đối tượng ưu tiên.

Để giải quyết các vấn đề trên, đồ án Hệ thống bãi giữ xe thông minh và dẫn đường (Smart Campus Parking) bằng ngôn ngữ C, tuân thủ nguyên lý phân tách độc lập giữa tầng giao diện người dùng và tầng xử lý logic cốt lõi. Sơ đồ bãi xe được mô hình hóa dưới dạng cấu trúc dữ liệu đồ thị có trọng số gồm tối đa 305 đỉnh, biểu diễn ma trận kề về khoảng cách giữa các cổng và 300 vị trí đỗ phân bố tại khu vực Giảng viên và Sinh viên. Trên cơ sở đó, thuật toán tìm đường ngắn nhất Dijkstra được áp dụng để tự động hóa việc tính toán và trích xuất lộ trình dẫn đường tối ưu từ cổng đến vị trí trống gần nhất cho từng nhóm đối tượng. Hoạt động kiểm soát xe ra/vào được tối ưu hóa thời gian truy xuất ở mức $O(1)$ thông qua cấu trúc Bảng băm, kết hợp cơ chế móc nối (Separate Chaining) để xử lý xung đột. Trong trường hợp bãi đỗ đạt mức sức chứa tối đa (maxAllowed), luồng phương tiện tại cổng sẽ được điều tiết tự động qua cấu trúc Hàng đợi theo nguyên lý FIFO.

Điểm nổi bật của hệ thống là khả năng xử lý linh hoạt bài toán chia sẻ không gian đỗ xe (phương tiện của sinh viên đỗ tạm tại khu vực giảng viên) thông qua Hàng đợi ưu tiên. Cấu trúc Min-Heap được cài đặt để phân loại dựa trên thời gian vào bãi, kết hợp cùng Ngăn xếp (Stack) nhằm mô phỏng thuật toán dời xe cản đường và ưu tiên giải phóng vị trí của phương tiện đỗ tạm có thời gian lưu bãi lâu nhất. Nhằm khắc phục các sai sót do nhiễu tín hiệu nhận diện từ camera hoặc sự cố mất thẻ, hệ thống tích hợp tính năng Tìm kiếm mờ (Fuzzy Search) ứng dụng khoảng cách Levenshtein. Thuật toán này cho phép truy xuất và đề xuất nhanh 3 biển số có độ tương đồng cao nhất với từ khóa truy vấn.

Kết quả của đồ án kiến tạo ra một ứng dụng cốt lõi hoạt động ổn định, quản lý bộ nhớ động chặt chẽ và đảm bảo tính toàn vẹn của luồng dữ liệu, tạo tiền đề vững chắc cho việc tích hợp lên các nền tảng giao diện đồ họa trực quan.

II. CƠ SỞ LÝ THUYẾT

2.1. Danh sách liên kết đơn

2.1.1. Khái niệm

Danh sách liên kết đơn là một cấu trúc dữ liệu tuyến tính động, được tạo thành từ tập hợp các phần tử nút. Các nút trong danh sách liên kết không nằm ở các vùng nhớ liên tiếp nhau mà phân tán ngẫu nhiên trong bộ nhớ Heap. Mỗi liên hệ thứ tự giữa các phần tử được duy trì thông qua các liên kết con trỏ một chiều.

2.1.2. Mã giả

```
Thuật toán Thêm_Xe_Vào_Đầu_Danh_Sách(Tham chiếu List l, Xe x):
    Đặt con trỏ p <- Tạo_Nút_Mới(x)    {Gọi hàm GetNode cấp phát động}
    Nếu p bằng NULL thì:
        Trả về thất bại
    Kết thúc nếu

    Nếu l.first bằng NULL thì:        {Trường hợp danh sách đang rỗng}
        l.first <- p
        l.last  <- p
    Ngược lại:                        {Danh sách đã có xe đỗ}
        p.next  <- l.first            {Nối đuôi nút mới vào nút đầu hiện hành}
        l.first <- p                  {Cập nhật nút đầu danh sách là nút mới}
    Kết thúc nếu
    Trả về thành công
    Kết thúc thuật toán
```

2.2. Ngăn xếp (Stack)

2.2.1. Khái niệm

Ngăn xếp là một cấu trúc vận hành theo cơ chế vào sau, ra trước. Dựa trên nguyên lý này, tiến trình bổ sung hoặc loại bỏ phần tử tuân thủ nghiêm ngặt quy tắc: phần tử được nạp vào cấu trúc muộn nhất sẽ là phần tử đầu tiên được phép giải phóng. Mặc dù hệ thống cho phép thêm dữ liệu vào ngăn xếp tại bất kỳ thời điểm nào, nhưng các thao tác truy xuất, đọc giá trị hoặc hủy bỏ trực tiếp chỉ được thực thi duy nhất đối với phần tử hiện hành đang nằm ở vị trí đỉnh của cấu trúc

2.2.2. Mã giả

```
Thuật toán Push(Tham chiếu Stack S, Xe x):
    { Đầu vào: Cấu trúc quản lý Ngăn xếp S, thông tin xe x cần lưu trữ }
    { Đầu ra: Ngăn xếp S được cập nhật thêm nút mới chứa xe x ở vị trí đỉnh }
}

// Bước 1: Cấp phát động một ô nhớ mới trong bộ nhớ RAM
Đặt con trỏ newNode <- Cấp phát động vùng nhớ kích thước StackNode
// Bước 2: Kiểm tra trạng thái bộ nhớ hệ thống
Nếu newNode bằng NULL thì:
    Thông báo lỗi "Lỗi hệ thống: Không đủ bộ nhớ RAM để cấp phát!"
```

```

    Trả về THẤT BẠI
    Kết thúc nếu
    // Bước 3: Nạp dữ liệu thực thể xe x vào nút mới
    newNode.xe <- x
    // Bước 4: Móc nối liên kết nút mới vào trên đỉnh Stack hiện tại
    newNode.next <- S.top { Cho nút mới liên kết tới nút đang ở đỉnh hiện
hành }
    // Bước 5: Cập nhật con trỏ đỉnh quản lý nút mới
    S.top <- newNode { Nút mới chính thức trở thành đỉnh Stack mới }
    Trả về THÀNH CÔNG
    Kết thúc thuật toán

```

2.3. Hàng đợi (Queue)

2.3.1. Khái niệm

Hàng đợi là một cấu trúc vận hành theo cơ chế vào trước, ra trước. Dựa trên nguyên lý này, tiến trình bổ sung hoặc loại bỏ phần tử tuân thủ quy tắc: phần tử được nạp vào cấu trúc sớm nhất sẽ là phần tử đầu tiên được phép giải phóng. Như vậy, thao tác xen vào hoặc nạp dữ liệu luôn được thực hiện tại vị trí cuối, trong khi thao tác trích xuất hoặc hủy bỏ dữ liệu chỉ được thực thi tại vị trí đầu của cấu trúc

2.3.2. Mã giả

```

Thuật toán EnQueue(Tham chiếu Queue Q, Xe x):
{ Đầu vào: Cấu trúc quản lý Hàng đợi Q, thực thể dữ liệu xe x cần thêm }
{ Đầu ra: Hàng đợi Q được cập nhật thêm nút mới chứa xe x ở cuối xâu }
// Bước 1: Cấp phát động một ô nhớ mới trong bộ nhớ RAM
Đặt con trỏ newNode <- Cấp phát động vùng nhớ kích thước QueueNode
// Bước 2: Kiểm tra trạng thái cấp phát của bộ nhớ hệ thống
Nếu newNode bằng NULL thì:
    Thông báo lỗi "Lỗi hệ thống: Không đủ bộ nhớ RAM để cấp phát!"
    Dừng thuật toán
Kết thúc nếu
// Bước 3: Đóng gói thông tin xe x và khóa mới liên kết kế tiếp
newNode.xe <- x
newNode.next <- NULL
// Bước 4: Đưa nút mới vào cuối hàng đợi dựa trên trạng thái rỗng
Nếu Q.rear bằng NULL (Hàng đợi đang rỗng) thì:
    Q.front <- newNode { Nút mới đóng vai trò vừa là đầu vừa là
cuối }
    Q.rear <- newNode
Ngược lại (Hàng đợi đã có sẵn các xe xếp hàng):
    Q.rear.next <- newNode { Móc nối nút cuối hiện hành tới nút mới }
    Q.rear <- newNode { Dịch chuyển con trỏ rear quản lý nút mới }
Kết thúc nếu
Kết thúc thuật toán

```

2.4. Bảng băm (Hash Table)

2.4.1. Khái niệm

Bảng băm (Hash Table) là một cấu trúc được thiết kế nhằm tối ưu hóa tốc độ thực thi của các thao tác chèn, tra cứu và loại bỏ các cặp khóa - giá trị. Cấu trúc này vận hành dựa trên

nguyên lý băm, sử dụng một hàm băm để chuyển đổi khóa thành một chỉ số tương ứng trong mảng. Chỉ số này định vị chính xác vị trí lưu trữ trong bộ nhớ (bucket) để liên kết trực tiếp với giá trị cần quản lý. Cơ chế này cho phép truy xuất trực tiếp đến phân vùng dữ liệu đích mà không cần thực hiện tiến trình duyệt tuần tự, từ đó tối ưu hóa độ phức tạp thời gian của các thao tác.

2.4.2. Mã giả

```
Thuật toán hashFunction(Chuỗi key):
{ Đầu vào: Chuỗi ký tự key đại diện cho Mã thẻ xe }
{ Đầu ra: Số nguyên index đại diện cho chỉ mục ô chứa (bucket) }
Đặt biến hash <- 0
Đặt biến i <- 0
// Duyệt qua từng ký tự của chuỗi cho đến khi gặp ký tự kết thúc chuỗi
Trong khi key[i] khác ký tự rỗng ('\0') thực hiện:
    hash <- (hash * 31) + mã_ASCII_của(key[i]) { Thuật toán băm đa thức
nhân cơ số 31 }
    i <- i + 1
Kết thúc vòng lặp
Trả về (hash chia lấy dư cho HASH_SIZE) { Phép toán Modulo giới
hạn chỉ mục }
Kết thúc thuật toán
```

2.5. Separate Chaining

2.5.1. Khái niệm

Separate Chaining là kỹ thuật giải quyết xung đột trong cấu trúc dữ liệu Bảng băm. Hiện tượng xung đột phát sinh khi hàm băm ánh xạ hai hoặc nhiều khóa phân biệt về cùng một chỉ số của mảng lưu trữ nền tảng. Thay vì tìm kiếm một vị trí trống khác trên mảng, phương pháp này xử lý xung đột bằng cách tổ chức mỗi phần tử của mảng (ô chứa) thành một con trỏ quản lý một danh sách liên kết độc lập. Khi xảy ra xung đột, phần tử mới sẽ được cấp phát bộ nhớ động và nối tiếp vào danh sách liên kết tại chỉ số mảng tương ứng, giúp bảo toàn các cặp khóa - giá trị mà không làm thay đổi hay chiếm dụng không gian lưu trữ của các vị trí khác trong bảng băm.

2.5.2. Mã giả

```
Thuật toán insertHash(Tham chiếu HashTable HT, Xe xe_moi):
// Bước 1: Tính chỉ mục ô chứa bằng cách băm Mã thẻ xe
Đặt số nguyên idx <- hashFunction(xe_moi.maThe)
// Bước 2: Cấp phát động một nút băm mới trong RAM
Đặt con trỏ newNode <- Cấp phát động vùng nhớ kích thước HashNode
Nếu newNode bằng NULL thì:
    Thông báo lỗi "Lỗi: Không đủ bộ nhớ RAM!"
    Dừng thuật toán
Kết thúc nếu
// Bước 3: Đóng gói dữ liệu xe mới vào nút băm
newNode.xe <- xe_moi
// Bước 4: Áp dụng kỹ thuật Separate Chaining (Chèn vào đầu danh sách
liên kết)
```

```

newNode.next <- HT.buckets[idx]      { Nút mới trở tới nút đầu hiện tại
của thùng idx }
HT.buckets[idx] <- newNode{ô idx cập nhật đầu danh sách mới là newNode }
Kết thúc thuật toán

```

2.6. Ma trận kề (Adjacency Matrix)

2.6.1. Khái niệm

Ma trận kề biểu diễn cấu trúc dữ liệu đồ thị trong bộ nhớ dưới dạng một mảng hai chiều (ma trận vuông) có kích thước $V \times V$, với V là tổng số lượng đỉnh của đồ thị. Trong ma trận kề, cả hàng và cột đều đại diện cho danh sách các đỉnh của đồ thị. Giá trị nằm tại ô giao giữa hàng i và cột j (ký hiệu là $matrix[i][j]$) sẽ biểu diễn thông tin liên quan đến cạnh nối từ đỉnh i tới đỉnh j .

2.6.2. Mã giả

```

Thuật toán loadMapFromFile(Tham chiếu Graph G, Chuỗi ten_file):
{ Đầu vào: Cấu trúc đồ thị G, tên file cấu hình bản đồ (ví dụ:
"map.txt") }
{ Đầu ra: Đồ thị G được điền đầy đủ dữ liệu phân khu và ma trận kề
khoảng cách }
// Bước 1: Mở tập tin văn bản ở chế độ chỉ đọc ("rt")
Đặt con trỏ file f <- Mở file ten_file ở chế độ "rt"
// Bước 2: Kiểm tra sự tồn tại của tập tin bản đồ
Nếu f bằng NULL thì:
    Thông báo lỗi "Lỗi hệ thống: Không thể mở tập tin bản đồ!"
    Dừng thuật toán
Kết thúc nếu
// Bước 3: Đọc dòng đầu tiên để xác định số lượng đỉnh (G.n) của sa bàn
Đọc từ file f ghi vào G.n
// Bước 4: Đọc thông tin phân loại khu vực cho từng đỉnh
Cho i chạy từ 0 đến G.n - 1 thực hiện:
    Đọc từ file f ghi vào G.zoneType[i]  {-1: Cổng, 1: Khu GV, 0: Khu
SV}
    G.isOccupied[i] <- 0      { Khởi tạo mặc định: tất cả ô đều TRỐNG }
Kết thúc vòng lặp
G.isOccupied <- 1           { Đỉnh 0 là Cổng vào, mặc định luôn khóa rào chắn }
// Bước 5: KHỞI TẠO MA TRẬN KỀ HAI CHIỀU (Đọc trọng số khoảng cách giữa
các đỉnh)
Cho i chạy từ 0 đến G.n - 1 thực hiện:
    Cho j chạy từ 0 đến G.n - 1 thực hiện:
        Đọc từ file f ghi vào G.matrix[i][j]  { Gán khoảng cách từ đỉnh
i tới đỉnh j }
        Kết thúc vòng lặp j
    Kết thúc vòng lặp i
// Bước 6: Đóng tập tin sau khi hoàn tất nạp dữ liệu vào RAM
Đóng file f
Kết thúc thuật toán

```

2.7. Hàng đợi ưu tiên (Priority Queue)

2.7.1. Khái niệm

Hàng đợi ưu tiên (Priority Queue) là cấu trúc trong đó mỗi phần tử lưu trữ đều được gắn liền với một giá trị ưu tiên. Khác với hàng đợi thông thường hoạt động nghiêm ngặt theo cơ chế FIFO (Vào trước ra trước), các phần tử trong hàng đợi ưu tiên được truy xuất và phục vụ hoàn toàn dựa trên mức độ ưu tiên của chúng:

- Các phần tử có mức độ ưu tiên cao hơn sẽ luôn được truy xuất hoặc loại bỏ ra khỏi hàng đợi trước các phần tử có mức độ ưu tiên thấp hơn, bất kể trình tự chúng được đưa vào hệ thống
- Khi một phần tử mới được thêm vào, cấu trúc sẽ tự động sắp xếp và chèn phần tử đó vào vị trí tương ứng với mức độ ưu tiên vốn có của nó

2.7.2. Mã giả

```
Thuật toán pushPQ(Tham chiếu PriorityQueue PQ, Xe xe_moi):
{ Đầu vào: Cấu trúc hàng đợi ưu tiên PQ, thực thể xe_moi cần chèn }
{ Đầu ra: Hàng đợi PQ được cập nhật, xe_moi được đưa về đúng vị trí vun
đồng }
// Bước 1: Kiểm tra dung lượng bộ nhớ mảng Heap
Nếu PQ.size bằng PQ.capacity thì:
    PQ.capacity <- PQ.capacity * 2    { Nhân đôi sức chứa tối đa }
    Cấp phát lại bộ nhớ (Reallocate) cho PQ.heap với sức chứa mới
    Nếu PQ.heap bằng NULL thì:
        Thông báo lỗi "Lỗi hệ thống: Không thể tái cấp phát bộ nhớ
Heap!"
        Dừng thuật toán
    Kết thúc nếu
    Kết thúc nếu
// Bước 2: Chèn tạm thời xe mới vào vị trí cuối cùng của mảng Heap
PQ.heap[PQ.size] <- xe_moi
Đặt số nguyên i <- PQ.size    { Lưu vết chỉ số hiện tại của xe mới }
PQ.size <- PQ.size + 1    { Tăng số lượng xe hiện hành }
// Bước 3: Thuật toán Vun đồng ngược lên (Up-Heapify / Shift-Up)
// Trong khi i chưa phải là gốc (i > 0) VÀ xe hiện tại vào bãi SÓM HƠN
xe cha của nó
    Trong khi (i > 0) VÀ (PQ.heap[i].thoiGianVao < PQ.heap[(i - 1) /
2].thoiGianVao) thực hiện:
        Hoán_Vị(PQ.heap[i], PQ.heap[(i - 1) / 2])    { Đổi chỗ xe con và xe
cha }
        i <- (i - 1) / 2    { Dịch chuyển chỉ số i lên vị trí nút cha }
    Kết thúc vòng lặp
Kết thúc thuật toán
```

2.8. Heap (Min Heap)

2.8.1. Khái niệm

Min-Heap là một cấu trúc dữ liệu phi tuyến tính dựa trên mô hình cây nhị phân hoàn chỉnh. Trong cấu trúc Min-Heap, phần tử tại mỗi nút cha luôn luôn có giá trị nhỏ hơn hoặc bằng giá trị lưu trữ tại các nút con trực tiếp của nó. Nhờ đặc tính ràng buộc này, phần tử có giá trị nhỏ nhất trong toàn bộ tập dữ liệu sẽ luôn nằm ở vị trí đỉnh gốc (root node) của cây

2.8.2. Mã giả

```
Thuật toán Min_Heapify(Mảng arr, Số nguyên r, Số nguyên i):
{ Đầu vào: Mảng arr, giới hạn phạm vi hiệu chỉnh r, chỉ số nút cần xét i
}
{ Đầu ra: Nhánh cây có gốc tại i được hiệu chỉnh thỏa mãn tính chất Min-Heap }
Đặt số nguyên smallest <- i    { Giả định nút cha hiện tại là nhỏ nhất }
Đặt số nguyên index <- 2 * i + 1 { Xác định chỉ số nút con bên trái }
// Bước 1: So sánh hai nút con để tìm nút con có giá trị nhỏ nhất
Nếu (index + 1 < r) VÀ (arr[index] > arr[index + 1]) thì:
    index <- index + 1 { Chọn nút con bên phải vì nó nhỏ hơn con trái }
Kết thúc nếu
// Bước 2: So sánh nút con nhỏ nhất vừa chọn với nút cha (smallest)
Nếu (index < r) VÀ (arr[index] < arr[smallest]) thì:
    smallest <- index { Cập nhật chỉ số của phần tử nhỏ nhất }
Kết thúc nếu
// Bước 3: Nếu nút cha lớn hơn nút con (vi phạm tính chất Min-Heap)
Nếu smallest khác i thì:
    Hoán_Vị(arr[i], arr[smallest]) { Đổi chỗ nút cha và nút con nhỏ nhất }
    Gọi đệ quy Min_Heapify(arr, r, smallest) { Lan truyền hiệu chỉnh xuống nhánh con }
Kết thúc nếu
Kết thúc thuật toán
```

2.9. Quy hoạch động (Dynamic Programming)

Quy hoạch động là một phương pháp thiết kế giải thuật vận hành bằng cách chia bài toán chính thành các bài toán con, tìm lời giải cho các bài toán con đó rồi tổng hợp lại để hình thành lời giải hoàn chỉnh. Để có thể áp dụng phương pháp này, bài toán cần giải quyết bắt buộc phải hội tụ đủ hai thuộc tính cấu trúc: Các bài toán con trùng lặp (lời giải của bài toán con này là một phần cấu thành nên lời giải của bài toán con khác) và Cấu trúc con tối ưu (lời giải tối ưu của bài toán chính được xây dựng từ lời giải tối ưu của các bài toán con nhỏ hơn).

2.10. Khoảng cách Levenshtein

2.10.1. Khái niệm

Khoảng cách Levenshtein là một thuật toán đo lường sai khác giữa hai chuỗi ký tự bằng cách xác định số lượng thao tác tối thiểu để biến đổi chuỗi ban đầu thành chuỗi mục tiêu. Các thao tác hợp lệ được định nghĩa ở cấp độ đơn ký tự, bao gồm ba phép biến đổi nguyên tử: chèn (bổ sung một ký tự), xóa (loại bỏ một ký tự), và thay thế (đổi một ký tự này thành một ký tự khác).

2.10.2. Mã giả

```
Thuật toán getLevenshteinDistance(Chuỗi s1, Chuỗi s2) bằng Quy hoạch động:
{ Đầu vào: Hai chuỗi ký tự s1 (độ dài M) và s2 (độ dài N) đại diện cho hai biến số }
{ Đầu ra: Số nguyên là khoảng cách chỉnh sửa tối thiểu giữa hai chuỗi biến số }
```

```

Đặt M <- Độ_Dài(s1)
Đặt N <- Độ_Dài(s2)
// Khởi tạo bảng phương án Quy hoạch động kích thước (M + 1) x (N + 1)
trong bộ nhớ RAM
Đặt mảng hai chiều dp kích thước (M + 1) x (N + 1) số nguyên
// Bước 1: Khởi tạo các bài toán con cơ sở (Base Cases)
Cho i chạy từ 0 đến M thực hiện:
    dp[i] <- i      { Biến đổi tiền tố s1[0..i-1] thành chuỗi rỗng bằng
phép xóa i ký tự }
    Kết thúc vòng lặp
    Cho j chạy từ 0 đến N thực hiện:
        dp[j] <- j      { Biến đổi chuỗi rỗng thành tiền tố s2[0..j-1]
bằng phép chèn j ký tự }
        Kết thúc vòng lặp
// Bước 2: Điền bảng phương án theo cơ chế từ dưới lên (Bottom-up
Tabulation)
Cho i chạy từ 1 đến M thực hiện:
    Cho j chạy từ 1 đến N thực hiện:

        // Xác định chi phí của phép thay thế ký tự (cost)
        Nếu s1[i - 1] bằng s2[j - 1] thì:
            Đặt cost <- 0  { Hai ký tự trùng khớp nhau, không tốn chi
phí thay đổi }
        Ngược lại:
            Đặt cost <- 1  { Ký tự khác nhau, tốn 1 phép thay thế }
        Kết thúc nếu
        // Áp dụng công thức chuyển trạng thái Quy hoạch động để tìm giá
trị tối ưu
        Đặt xoa <- dp[i - 1][j] + 1      { Phép xóa 1 ký tự }
        Đặt chen <- dp[i][j - 1] + 1     { Phép chèn 1 ký tự }
        Đặt thay_the <- dp[i - 1][j - 1] + cost {Phép thay thế 1 ký tự }
        // Lưu kết quả tối ưu của bài toán con vào ô nhớ tương ứng
        dp[i][j] <- Giá_Trị_Nhỏ_Nhất_Trong_Ba(xoa, chen, thay_the)
        Kết thúc vòng lặp j
    Kết thúc vòng lặp i
    Trả về dp[M][N]      { Kết quả tối ưu của bài toán gốc cần tìm }

Kết thúc thuật toán

```

2.11. Thuật toán Bubble Sort

2.11.1. Khái niệm

Bubble Sort là thuật toán sắp xếp bằng so sánh và đổi chỗ các cặp phần tử kế cận nhau nếu chúng đứng sai thứ tự (nghịch thế). Thao tác này giúp đưa phần tử nhỏ (hoặc lớn) hơn trong cặp đang xét về vị trí đứng đầu (hoặc cuối) dãy hiện hành. Thuật toán lặp lại xử lý trên cho đến khi không còn cặp phần tử nào vi phạm. Sau mỗi lượt duyệt, các phần tử tối ưu tiếp theo sẽ lần lượt được nổi lên về đúng vị trí của chúng, đồng thời phạm vi dãy cần xét ở lượt kế tiếp sẽ được co ngắn lại.

2.11.2. Mã giả

```

Thuật toán BubbleSortFuzzy(Mảng arr, Số nguyên n):
    { Đầu vào: Mảng arr gồm n phần tử kiểu FuzzyResult cần sắp xếp }

```

```

{ Đầu ra: Mảng arr được sắp xếp theo thứ tự score tăng dần (độ lệch nhỏ nhất lên đầu) }

Đặt số nguyên i <- 0
Đặt số nguyên j <- 0

// Vòng lặp ngoài co ngắn dần phân vùng cần sắp xếp từ đầu mảng [2]
Cho i chạy từ 0 đến n - 2 thực hiện:

    // Vòng lặp trong duyệt ngược từ cuối dãy về vị trí i hiện hành [2]
    Cho j chạy từ n - 1 giảm dần về i + 1 thực hiện:

        // Nếu phát hiện nghịch thế: xe đứng sau có độ lệch ký tự
(score) nhỏ hơn xe đứng trước
        Nếu arr[j].score < arr[j - 1].score thì:

            Hoán_Vị(arr[j], arr[j - 1])    { Đổi chỗ hai thực thể kết quả }

        Kết thúc nếu

            Kết thúc vòng lặp j
        Kết thúc vòng lặp i
    Kết thúc thuật toán

```

2.12. Thuật toán Fuzzy Search

2.12.1. Khái niệm

Fuzzy Search hay so khớp chuỗi xấp xỉ dùng để nhận diện các phần tử văn bản, chuỗi ký tự hoặc mục nhập có mức độ tương đồng cao nhưng không nhất thiết phải trùng khớp hoàn toàn. Phương pháp này được áp dụng phổ biến khi hệ thống không thể tìm thấy kết quả khớp chính xác tuyệt đối cho một từ khóa hay truy vấn trong cơ sở dữ liệu. Lúc này, thuật toán sẽ đối chiếu và trả về các kết quả có tỷ lệ tương đồng lớn hơn một ngưỡng được xác định trước.

2.12.2. Mã giả

```

Thuật toán fuzzySearchLicensePlate(HashTable HT, Chuỗi queryPlate):
{ Đầu vào: Bảng băm HT chứa các xe đỗ trong bãi, chuỗi truy vấn queryPlate }
{ Đầu ra: Mảng chứa tối đa 3 chiếc xe có độ tương đồng biến số cao nhất }

// Bước 1: Chuẩn hóa chuỗi truy vấn đầu vào (viết hoa, loại bỏ ký tự đặc biệt)
Đặt chuỗi normQuery
Gọi normalizeString(queryPlate, normQuery)
// Bước 2: Khởi tạo danh sách ứng viên tạm để chứa kết quả khớp
Đặt mảng candidateList gồm 1000 phần tử kiểu FuzzyResult
Đặt số nguyên count <- 0
// Bước 3: Duyệt toàn bộ bảng băm (Separate Chaining) để tìm xe
Cho i chạy từ 0 đến HASH_SIZE - 1 thực hiện:
    Đặt con trỏ chạy curr <- HT.buckets[i]
    Trong khi curr khác NULL thực hiện:
        // Chuẩn hóa biến số của chiếc xe đang xét trong bãi

```

```

        Đặt chuỗi normPlate
        Gọi normalizeString(curr.xe.bienSo, normPlate)

        // Tính điểm số sai lệch bằng thuật toán khoảng cách Levenshtein
        Đặt số nguyên dist <- getLevenshteinDistance(normQuery,
normPlate)

        // Ghi nhận ứng viên vào danh sách tạm
        candidateList[count].xe <- curr.xe
        candidateList[count].score <- dist
        count <- count + 1
        curr <- curr.next
        Kết thúc vòng lặp
    Kết thúc vòng lặp
    // Bước 4: Sắp xếp danh sách ứng viên theo score tăng dần (lệch ít nhất
lên đầu)
    Gọi BubbleSortFuzzy(candidateList, count)
    // Bước 5: Trích xuất ra tối đa 3 chiếc xe có độ tương đồng cao nhất
    Đặt mảng topResults gồm 3 phần tử kiểu Xe
    Đặt số nguyên limit <- Giá_Trị_Nhỏ_Nhất(3, count)
    Cho i chạy từ 0 đến limit - 1 thực hiện:
        topResults[i] <- candidateList[i].xe
    Kết thúc vòng lặp
    Trả về topResults
Kết thúc thuật toán

```

2.13. Thuật toán Shortest Path Dijkstra

2.13.1. Khái niệm

Trong lý thuyết đồ thị, bài toán tìm đường đi ngắn nhất là bài toán tìm kiếm một lộ trình di chuyển giữa hai đỉnh trên một đồ thị có trọng số sao cho tổng trọng số của các cạnh cấu thành nên lộ trình đó là nhỏ nhất. Tổng trọng số tích lũy dọc theo các cạnh của lộ trình di chuyển được gọi là chi phí đường đi hoặc trọng số đường đi. Đồ thị biểu diễn bài toán này có thể mô phỏng rất nhiều mô hình thực tế, trong đó các đỉnh đại diện cho các địa điểm giao lộ, ô đỗ xe, hoặc bộ định tuyến, còn các cạnh đại diện cho các con đường kết nối vật lý

2.13.2. Mã giả

```

Thuật toán findNearestSlot(Đồ thị G, Số nguyên startNode, Số nguyên
isGiangVien, Tham chiếu Số nguyên pathLength):
    { Đầu vào: Đồ thị G (chứa ma trận kề matrix, thông tin khu vực zoneType,
trạng thái isOccupied) }
    {
        startNode (Cổng vào = 0), isGiangVien (Phân khu yêu cầu: 1:
Khu GV, 0: Khu SV) }
    { Đầu ra: Mảng chứa lộ trình di chuyển tối ưu từ cổng đến ô đỗ gần nhất
    }

    // Bước 1: Khởi tạo các mảng hỗ trợ lưu trữ trạng thái duyệt
    Đặt mảng dist gồm G.n phần tử kiểu Số nguyên        { Lưu khoảng cách
ngắn nhất tạm thời từ nguồn }
    Đặt mảng visited gồm G.n phần tử kiểu Logic            { Đánh dấu các đỉnh đã
được duyệt xong }
    Đặt mảng trace gồm G.n phần tử kiểu Số nguyên        { Lưu vết đỉnh cha
đứng ngay trước (tiền bối) }

```

```

    Cho i chạy từ 0 đến G.n - 1 thực hiện:
        dist[i] <- INF          { Mặc định khoảng cách ban đầu tới tất cả các
đỉnh là vô cùng }
        visited[i] <- False    { Tất cả các đỉnh đều được đánh dấu chưa
duyet }
        trace[i] <- -1         { Chưa ghi nhận vết di chuyển }
    Kết thúc vòng lặp

    dist[startNode] <- 0       { Thiết lập khoảng cách từ cổng tới chính nó
bằng 0 }
    Đặt số nguyên targetNode <- -1 { Biến tạm chứa ID ô trống tối ưu được chọn
}

// Bước 2: Vòng lặp chính của thuật toán Dijkstra (Duyệt tối đa G.n lần)
Lặp G.n lần thực hiện:
    // 2.1. Tìm đỉnh u chưa duyệt có khoảng cách ngắn nhất đến nguồn
    Đặt số nguyên min_dist <- INF
    Đặt số nguyên u <- -1

    Cho i chạy từ 0 đến G.n - 1 thực hiện:
        Nếu (visited[i] bằng False) VÀ (dist[i] < min_dist) thì:
            min_dist <- dist[i]
            u <- i
        Kết thúc nếu
    Kết thúc vòng lặp

    // Nếu không tìm thấy đỉnh u thích hợp hoặc các đỉnh còn lại đều
không thể chạm tới
    Nếu (u bằng -1) HOẶC (dist[u] bằng INF) thì:
        Thoát vòng lặp chính
        Kết thúc nếu

    // Đánh dấu đỉnh u đã hoàn tất xử lý (visited)
    visited[u] <- True

    // CƠ CHẾ DỪNG SỚM CHỈ CÓ TRONG ĐỒ ẮN
    // Nếu đỉnh u được chọn là một ô trống TRỐNG và thuộc đúng phân khu quy
định
    Nếu (u khác startNode) VÀ (G.isOccupied[u] bằng 0) VÀ (G.zoneType[u]
bằng isGiangVien) thì:
        targetNode <- u      { Ghi nhận u chính là ô trống gần cổng
nhất }
        Thoát vòng lặp chính { Dừng thuật toán sớm để tiết kiệm chi phí
tính toán }
        Kết thúc nếu
    // 2.2. Tiến hành nói lỏng (Relaxation) khoảng cách tới các đỉnh v
kề với u
    Cho v chạy từ 0 đến G.n - 1 thực hiện:
        // Nếu tồn tại đường đi trực tiếp giữa u và v và v chưa duyệt
xong
        Nếu (G.matrix[u][v] khác INF) VÀ (visited[v] bằng False) thì:

            Đặt số nguyên alt <- dist[u] + G.matrix[u][v]
            // Nếu tìm được lộ trình mới qua u ngắn hơn khoảng cách cũ
đã lưu của v
            Nếu alt < dist[v] thì:
                dist[v] <- alt
                trace[v] <- u { Lưu vết: Đỉnh đứng trước v trong lộ
trình ngắn nhất là u }
            Kết thúc nếu

```

```

        Kết thúc nếu
        Kết thúc vòng lặp
    Kết thúc vòng lặp

    // Bước 3: Rebuild lộ trình di chuyển (Path Tracing) bằng cách đi lùi
    mảng tiền bối
    Nếu targetNode khác -1 thì:
        Đặt mảng path gồm G.n phần tử kiểu Số nguyên
        Đặt số nguyên temp <- targetNode
        pathLength <- 0
        Trong khi temp khác -1 thực hiện:
            path[pathLength] <- temp
            pathLength <- pathLength + 1
            temp <- trace[temp]      { Đi ngược về đỉnh cha đứng trước }
        Kết thúc vòng lặp
        Trả về path  { Trả về mảng chứa lộ trình ngược (Đích -> Cổng) }
    Ngược lại:
        Trả về NULL  { Báo lỗi: Không tìm thấy ô trống phù hợp }
    Kết thúc nếu
    Kết thúc thuật toán

```

2.14. Đồ thị (Graph)

2.14.1. Khái niệm

Đồ thị (Graph) là một cấu trúc dùng để mô hình hóa mối quan hệ giữa các đối tượng, bao gồm hai thành phần cơ bản gồm các đỉnh (các đối tượng) và các cạnh (liên kết giữa các cặp đỉnh, đại diện cho mối quan hệ hoặc đường kết nối giữa các đối tượng). Đồ thị cho phép tồn tại nhiều lộ trình và con đường khác nhau để di chuyển từ đỉnh này sang đỉnh khác, khác biệt hoàn toàn với tính chất tuần tự tuyến tính một chiều của mảng hay danh sách liên kết.

2.14.2. Mã giả

```

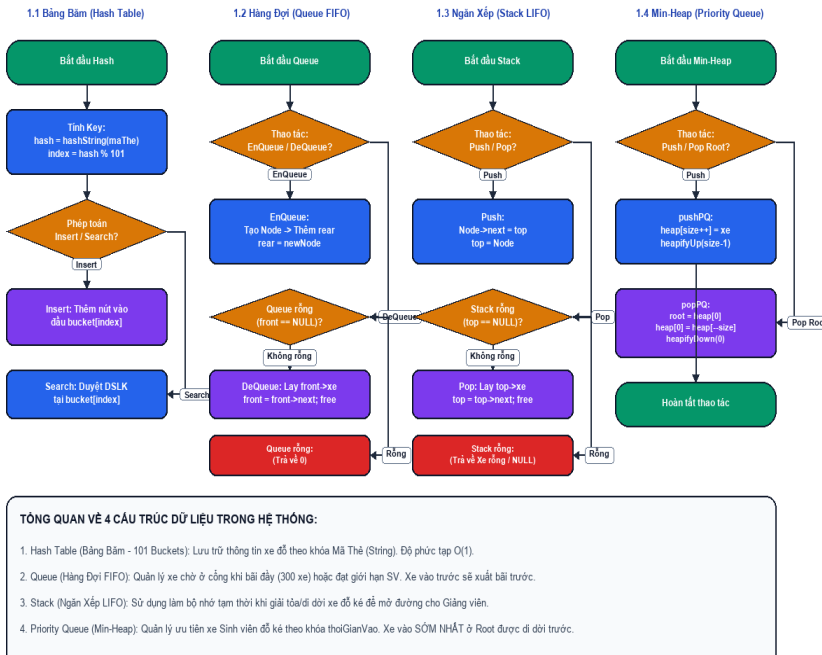
Thuật toán InitGraph(Tham chiếu Graph G, Số nguyên so_dinh):
    { Đầu vào: Đồ thị G cần cấu hình, Số lượng đỉnh so_dinh thực tế trên sa
    bàn }
    { Đầu ra: Đồ thị G được dọn dẹp và sẵn sàng kết nối các cạnh }

    G.n <- so_dinh

    Cho i chạy từ 0 đến G.n - 1 thực hiện:
        Cho j chạy từ 0 đến G.n - 1 thực hiện:
            Nếu i bằng j thì:
                G.matrix[i][j] <- 0      { Khoảng cách từ một điểm tới chính
                nó luôn bằng 0 }
            Ngược lại:
                G.matrix[i][j] <- INF    { Mặc định chưa có lối đi nào nối
                trực tiếp giữa i và j }
            Kết thúc nếu
        Kết thúc vòng lặp j
    Kết thúc vòng lặp i
    Kết thúc thuật toán

```

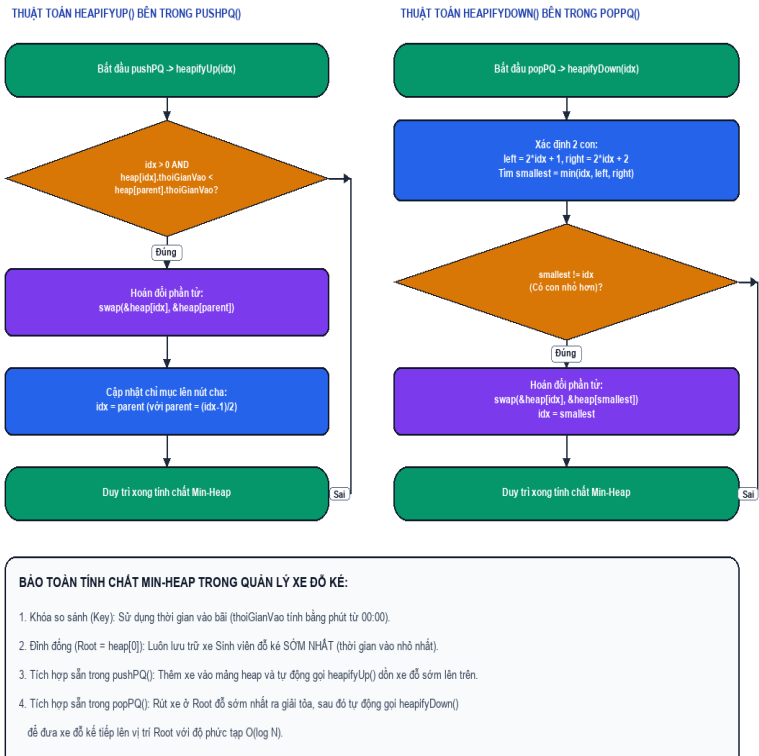
III. MÔ HÌNH HOẠT ĐỘNG



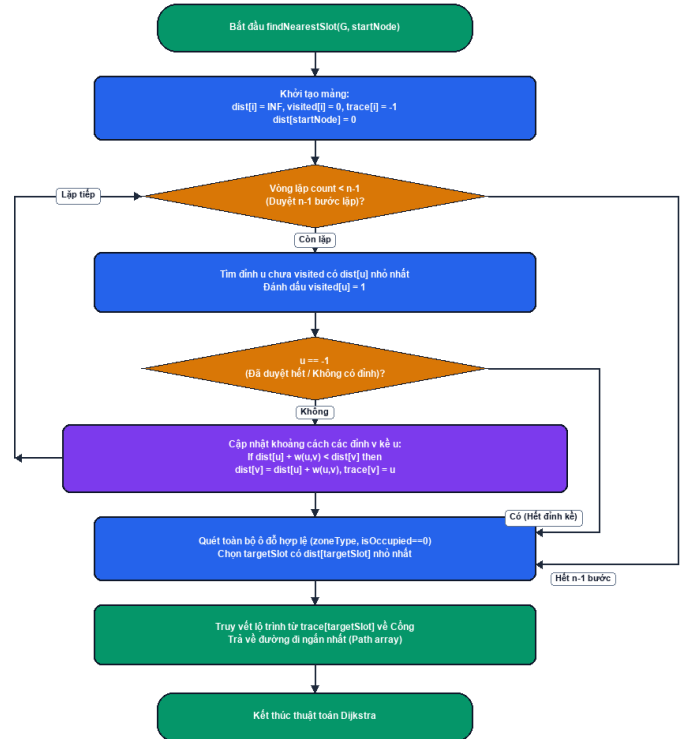
Hình 1. Lưu đồ giải thuật các cấu trúc dữ liệu cơ sở (HashTable, Queue FIFO, Stack LIFO, Min-Heap)



Hình 2. Lưu đồ giải thuật hàm InsertHash



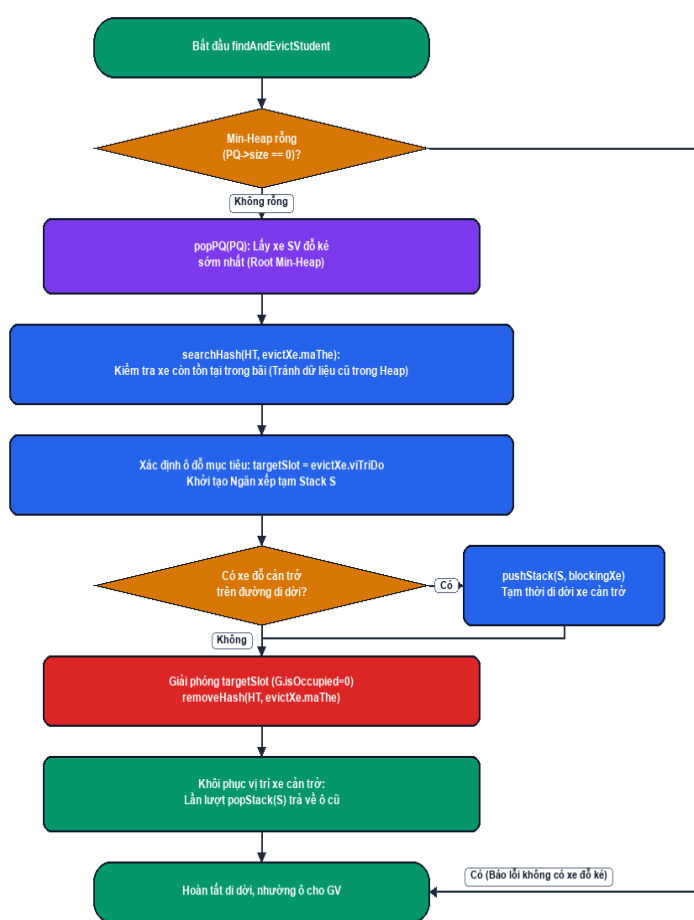
Hình 3. Lưu đồ giải thuật heapifyUp và heapifyDown



Hình 4. Lưu đồ giải thuật findNearestSlot



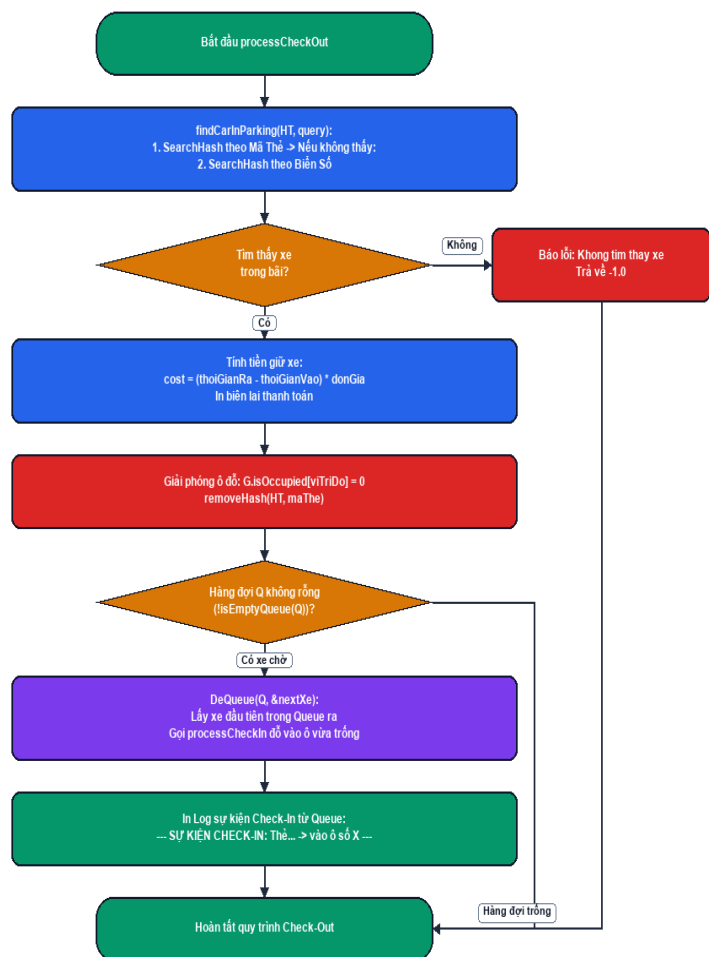
Hình 5. Lưu đồ giải thuật quy trình xe vào bãi (processCheckIn)



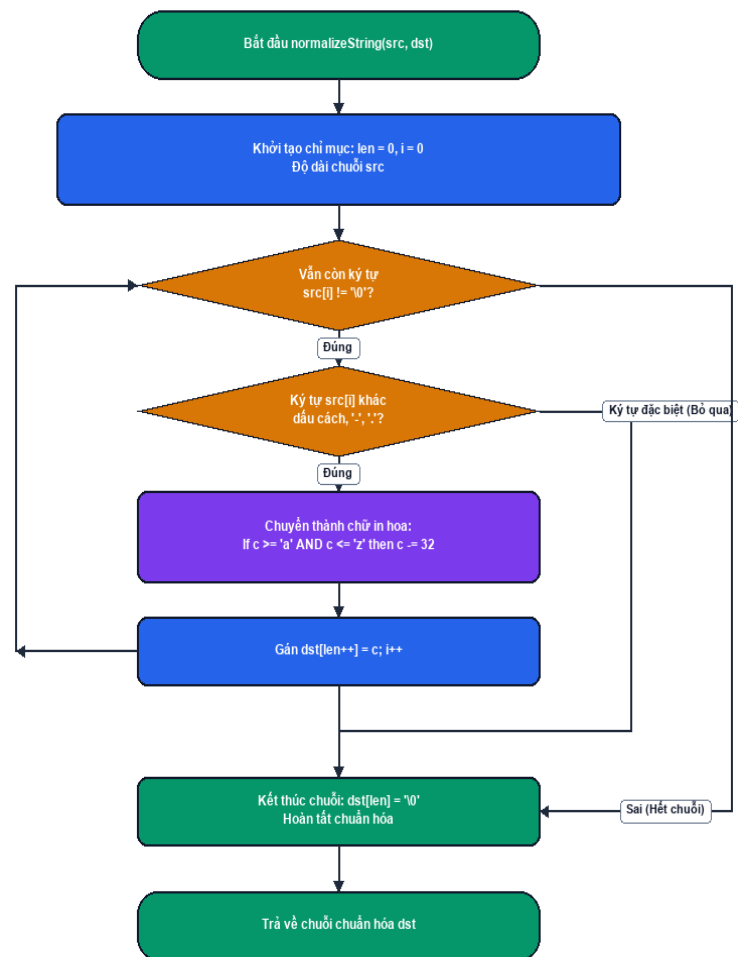
Hình 6. Lưu đồ giải thuật findAndEvictStudent



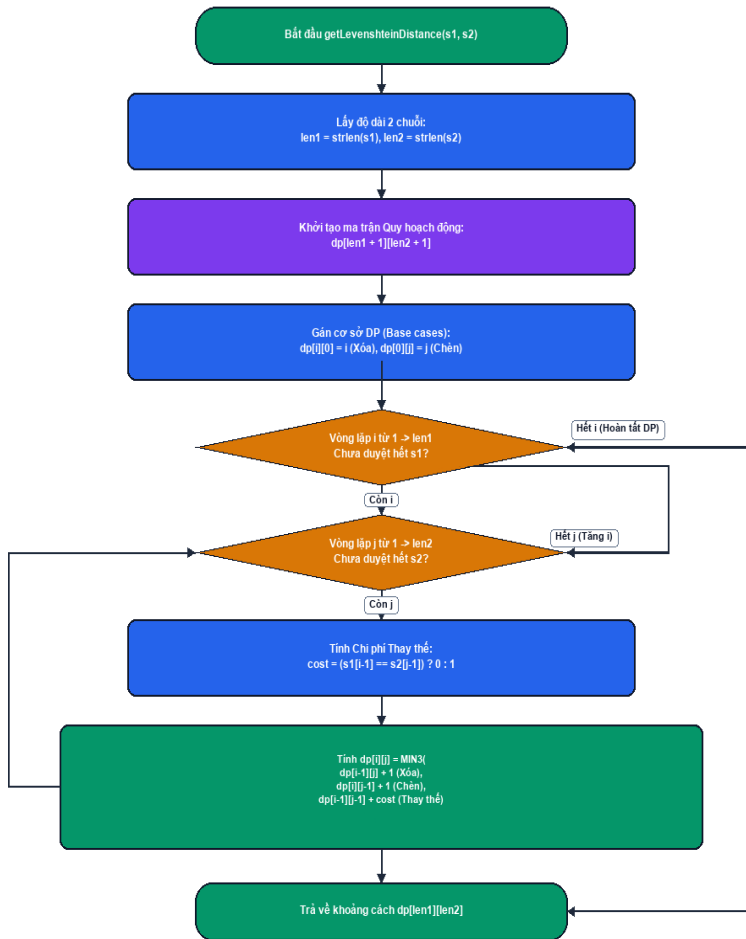
Hình 7. Lưu đồ giải thuật fuzzySearchLicensePlate



Hình 8. Lưu đồ giải thuật processCheckOut

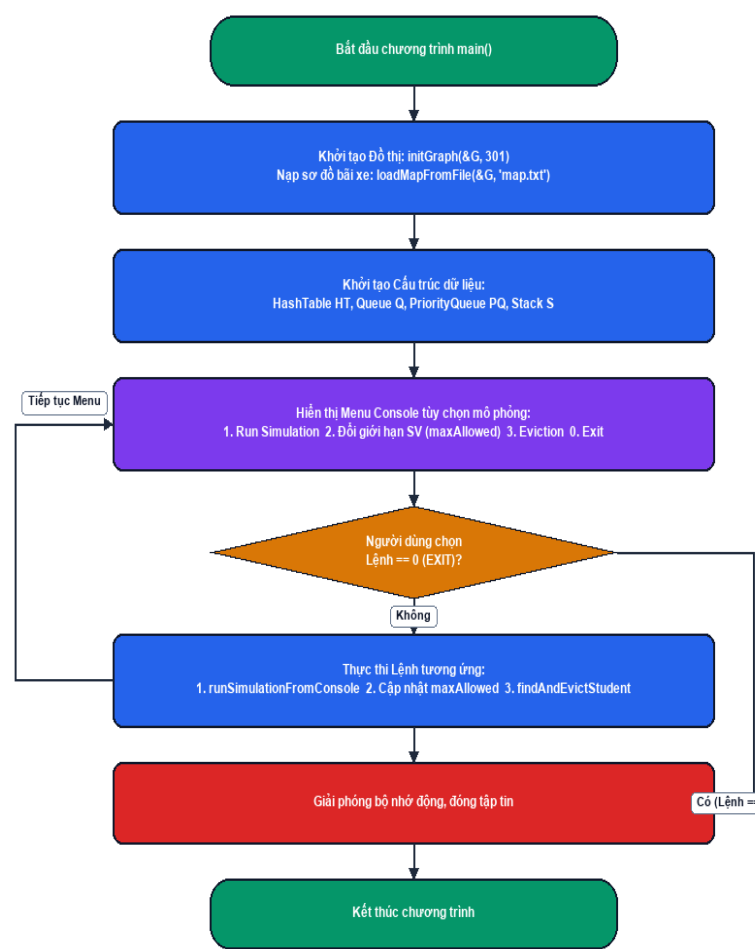


Hình 9. Lưu đồ giải thuật normalizeString

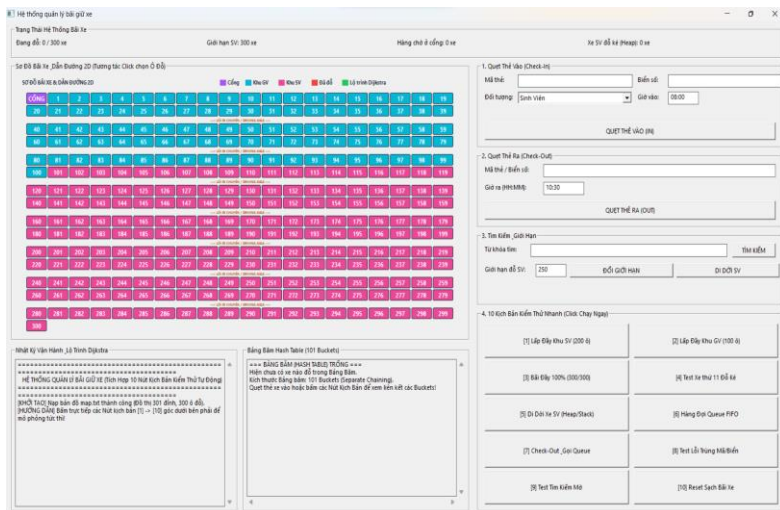


Hình 10. Lưu đồ giải thuật getLevenshteinDistance

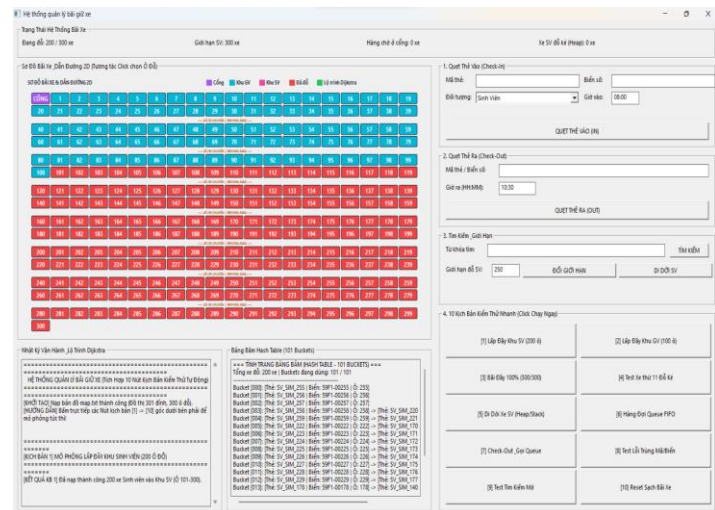
IV. KẾT QUẢ



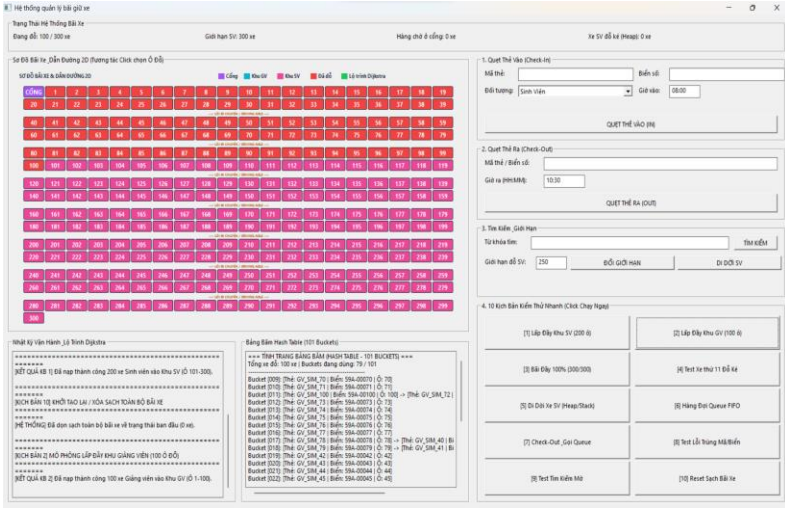
Hình 11. Lưu đồ giải thuật main()



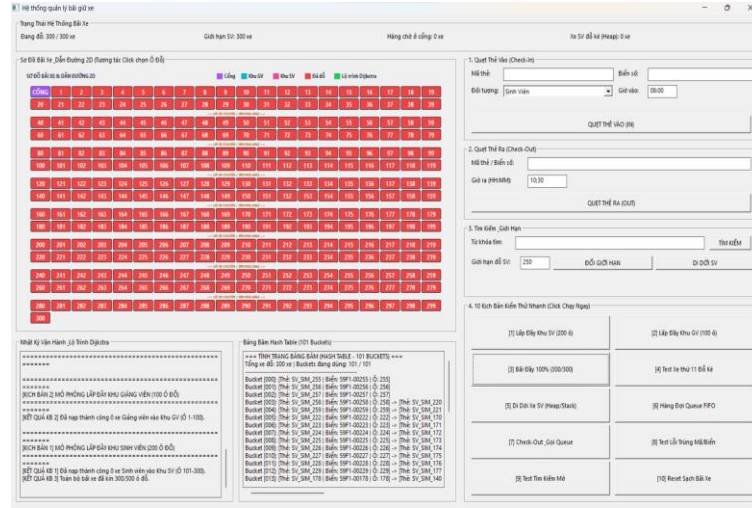
Hình 12. Giao diện tổng thể



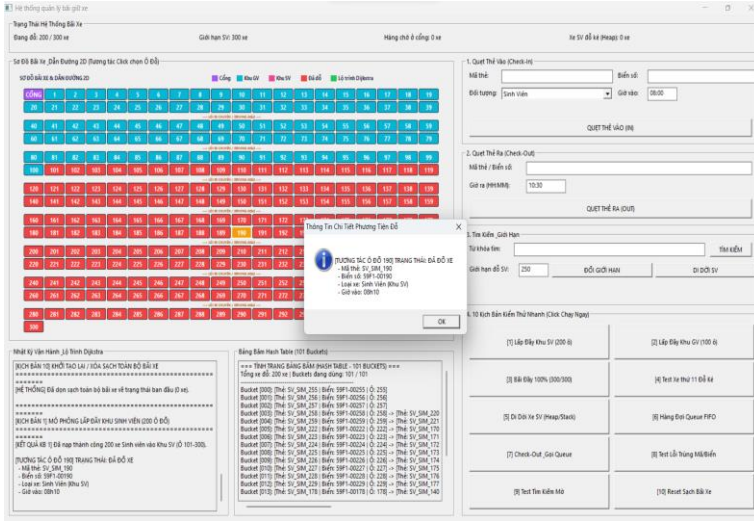
Hình 13. Kết quả khi bãi đỗ sinh viên được lấp đầy



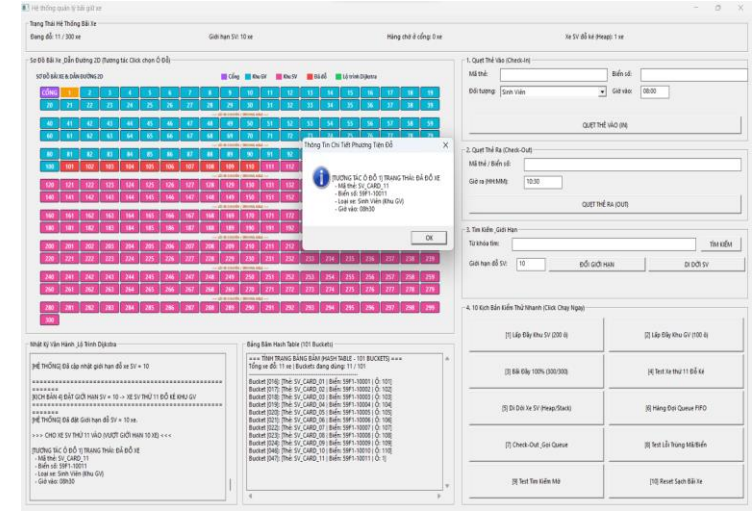
Hình 14. Kết quả khi bãi đỗ Giảng viên được lắp đầy



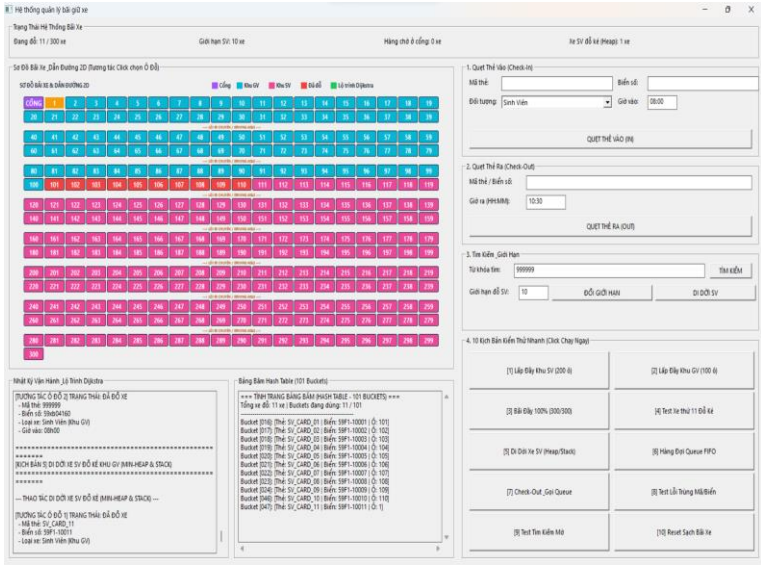
Hình 15. Kết quả khi bãi xe được lắp đầy



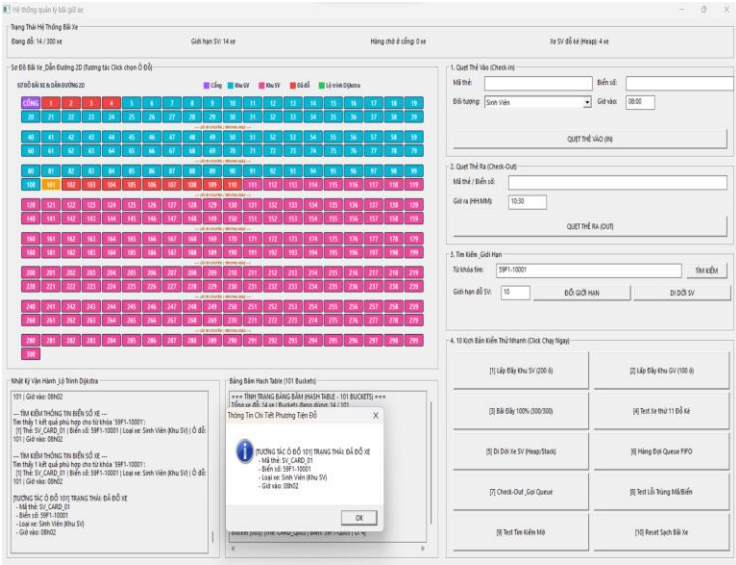
Hình 16. Khi bấm vào 1 ô bất kì thì sẽ có thông tin ô đỗ bao gồm mã thẻ, biển số xe, loại xe, khu đỗ và thời gian vào



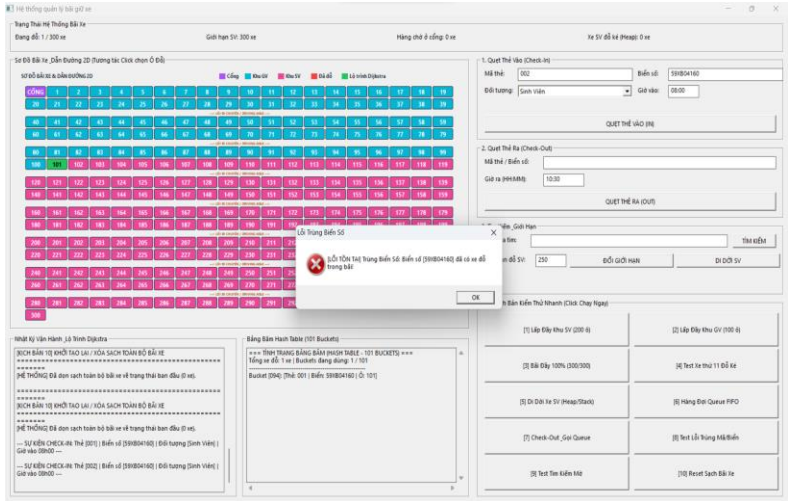
Hình 17. Khi bãi xe sinh viên đầy thì xe sinh viên đậu vào bãi của GV



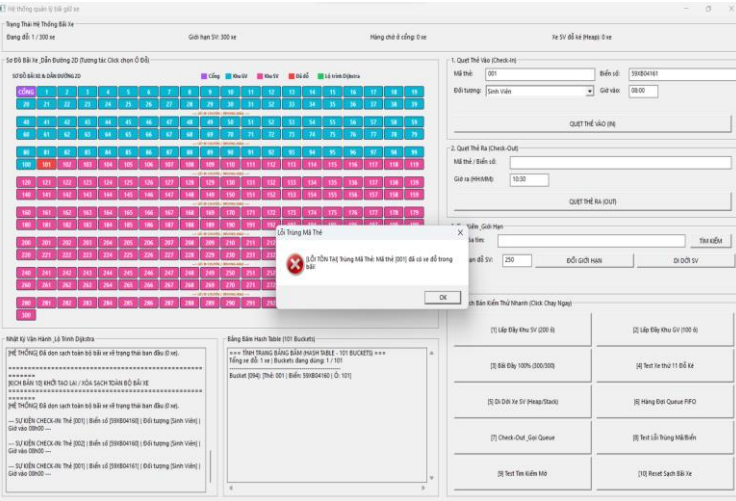
Hình 18. Trường hợp bãi xe Giảng viên đã đầy thì phải di dời xe SV đầu nhờ để cho Giảng viên khác vào



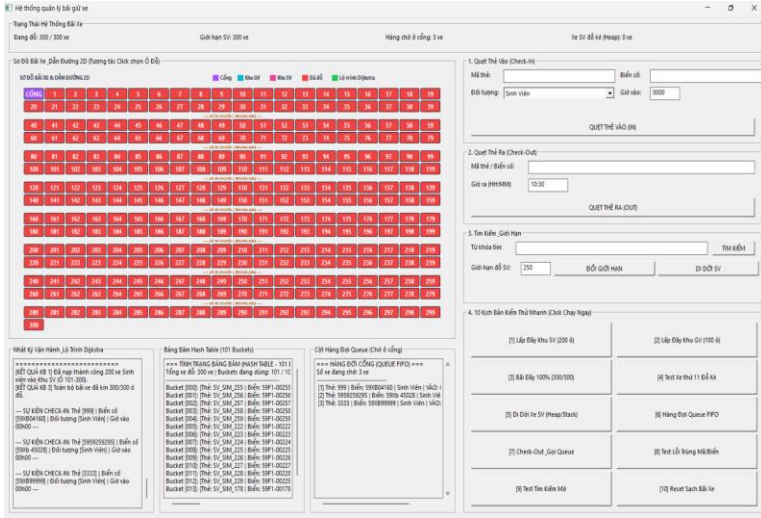
Hình 19. Tìm kiếm thông qua biên số xe hoặc mã số thẻ



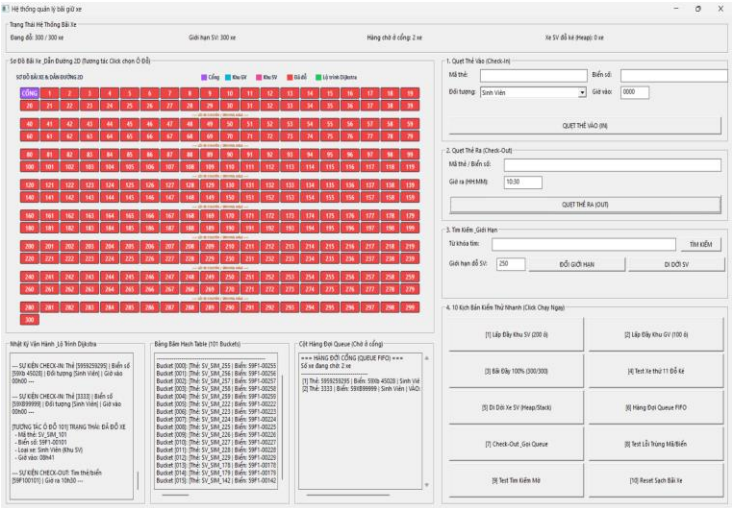
Hình 20. Thông báo trùng biên số



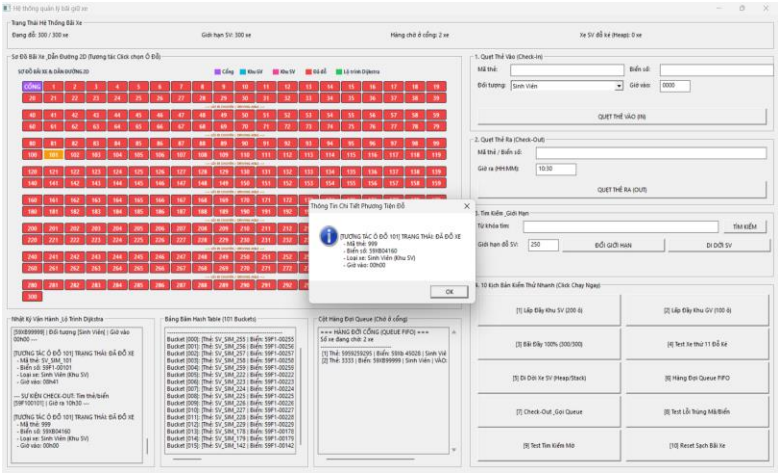
Hình 21. Thông báo trùng mã thẻ



Hình 22. Khi bãi xe đã đầy thì xe vào sau sẽ được đỗ tạm bên ngoài bãi xe



Hình 23. Khi có một xe trong bãi ra khỏi bãi đỗ thì xe trong hàng đợi sẽ được đưa vào bãi xe theo FIFO



Hình 24. Xe mang biển số 59XB04160 vào ô 101

IV. PHÂN TÍCH - ĐÁNH GIÁ

Bảng băm được khởi tạo với một mảng tĩnh gồm 101 ô chứa (buckets). Vì mảng tĩnh không có khả năng tự động mở rộng kích thước, hiện tượng xung đột bộ nhớ sẽ xảy ra khi hàm băm ánh xạ từ hai mã thẻ xe trở lên vào cùng một chỉ số. Để giải quyết tình trạng này, Separate Chaining được tích hợp nhằm quản lý các điểm xung đột. Hệ thống có thể móc nối các phương tiện bị trùng lặp mã băm thành một chuỗi độc lập tại cùng một ô chứa, qua đó bảo toàn tính toàn vẹn của dữ liệu. Danh sách liên kết đơn đóng vai trò là cấu trúc nền tảng hỗ trợ cấp phát bộ nhớ động cho các phần tử. Separate Chaining vận hành như một phương pháp xử lý xung đột chuyên biệt. Bảng băm đóng vai trò là kho lưu trữ dữ liệu trung tâm, tối ưu hóa thời gian tra cứu thông tin. Cơ sở dữ liệu này không chỉ

lưu trữ trạng thái xe mà còn cung cấp các thông số định lượng như tổng số xe hiện hành (thông qua hàm `getCurrentCarCount`), làm dữ liệu đầu vào cần thiết cho việc kiểm tra sức chứa và tra cứu của các module liên quan trong hệ thống.

Cấu trúc Hàng đợi được triển khai thông qua danh sách liên kết với hai con trỏ quản lý front (đầu) và rear (cuối), đảm nhận nhiệm vụ kiểm soát phương tiện tại cổng vào. Cấu trúc này duy trì sự tương tác liên tục với Bảng bấm để đánh giá sức chứa hiện tại của hệ thống. Trong trường hợp Bảng bấm xác nhận tổng số phương tiện đã đạt ngưỡng giới hạn vật lý tối đa (300 xe) hoặc vượt quá hạn mức cho phép đối với nhóm sinh viên (`maxAllowed`), hệ thống sẽ tự động kích hoạt hàm `EnQueue` để đưa phương tiện vào trạng thái chờ. Ngược lại, khi có sự kiện xuất bãi làm trống vị trí đỗ, hệ thống sẽ thực thi hàm `DeQueue` để lấy phương tiện đầu tiên ra khỏi hàng đợi và tiếp tục đưa vào quy trình phân bổ chỗ đỗ. Cấu trúc Hàng đợi hoạt động như một vùng đệm an toàn để điều tiết giao thông, giúp ngăn chặn rủi ro quá tải và lỗi tràn bộ nhớ cho không gian sa bàn.

Không gian vật lý của bãi đỗ được số hóa vào cấu trúc Đồ thị thông qua ba mảng dữ liệu chính: `zoneType` (phân loại khu vực dành cho Giảng viên hoặc Sinh viên), `isOccupied` (quản lý trạng thái trống hoặc đã có xe của từng vị trí), và Ma trận kề `matrix` (lưu trữ trọng số khoảng cách giữa tối đa 305 đỉnh). Thuật toán Dijkstra (được triển khai qua hàm `findNearestSlot`) phụ thuộc hoàn toàn vào cấu trúc dữ liệu này để có thể vận hành. Cụ thể, thuật toán trực tiếp truy xuất Ma trận kề để tính toán và cộng dồn trọng số đường đi ($\text{dist}[u] + G.\text{matrix}[u][v]$). Đồng thời, các mảng `isOccupied` và `zoneType` cung cấp những tham số bắt buộc để thuật toán đối chiếu và kích hoạt cơ chế dừng sớm (Early Stop), cho phép quá trình duyệt đỉnh kết thúc ngay khi xác định được một ô đỗ thỏa mãn đồng thời điều kiện còn trống và đúng đối tượng phân khu. Đồ thị và Ma trận kề đóng vai trò thiết lập nền tảng không gian, các ràng buộc vật lý và mạng lưới kết nối, hoạt động tương tự như một bản đồ số hóa của toàn bộ bãi giữ xe. Trong khi đó, thuật toán Dijkstra đảm nhận vai trò là hệ thống điều hướng cốt lõi, thực hiện việc tính toán và trích xuất lộ trình di chuyển ngắn nhất, tối ưu nhất từ vị trí Cổng (`startNode = 0`) đến không gian đỗ xe hợp lệ gần nhất.

Trong trường hợp thuật toán định tuyến (Dijkstra) buộc phải phân bổ phương tiện của sinh viên đỗ tạm vào phân khu dành cho Giảng viên do tình trạng quá tải không gian, hệ thống sẽ lập tức truyền dữ liệu của phương tiện này đến Hàng đợi ưu tiên (Priority Queue) thông qua hàm pushPQ để ghi nhận sự tồn tại của xe đỗ nhờ. Cấu trúc lõi vận hành bên trong Hàng đợi ưu tiên là Min-Heap (được xây dựng trên mảng động heap), chịu trách nhiệm thực thi thuật toán vun đống (Heapify) liên tục mỗi khi có dữ liệu mới. Nhờ cơ chế này, phương tiện của sinh viên có thời gian vào bãi sớm nhất (giá trị thời gian vào bãi nhỏ nhất) sẽ luôn được ưu tiên đẩy lên vị trí gốc (Root) của cấu trúc dữ liệu. Khi một Giảng viên tiến vào bãi nhưng phân khu đã kín chỗ, hệ thống sẽ gọi hàm popPQ để trích xuất chính xác phương tiện mục tiêu cần bị giải tỏa từ đỉnh của Min-Heap. Tại thời điểm này, Ngăn xếp (Stack) bắt đầu tham gia vào quy trình thực thi: hệ thống tiến hành duyệt Đồ thị từ ô đỗ số 1 đến vị trí ngay trước xe mục tiêu, đồng thời sử dụng lệnh Push để đưa tạm thời các phương tiện cản đường vào bộ nhớ LIFO của Ngăn xếp nhằm dọn đường. Ngay sau khi phương tiện mục tiêu được di dời khỏi bãi đỗ, Ngăn xếp sẽ sử dụng vòng lặp chứa lệnh Pop để truy xuất và khôi phục toàn bộ các phương tiện cản đường về đúng vị trí ban đầu trên Đồ thị.

Trong trường hợp quá trình Check-out gặp sự cố do dữ liệu biên số cung cấp bị sai lệch (so khớp chính xác thất bại), thuật toán tìm kiếm mờ (Fuzzy Search) sẽ được kích hoạt để quét toàn bộ dữ liệu hiện có trong Bảng băm nhằm thu thập các chuỗi biên số xe ứng viên. Các chuỗi ký tự này sau đó được chuyển qua hàm Khoảng cách Levenshtein để tính toán và đánh giá mức độ sai lệch. Nhằm tối ưu hóa tốc độ thực thi của thuật toán Levenshtein, hệ thống áp dụng phương pháp Quy hoạch động (Dynamic Programming) thông qua việc khởi tạo mảng hai chiều dp. Cấu trúc này giúp chia nhỏ bài toán so khớp từng ký tự, từ đó tính toán chính xác số thao tác biến đổi tối thiểu (bao gồm các phép chèn, xóa và thay thế). Những phương tiện có điểm số sai lệch thỏa mãn điều kiện ngưỡng ($\text{Score} \leq 3$) sẽ được lưu vào một danh sách ứng viên. Tại bước xử lý cuối cùng, thuật toán Sắp xếp nổi bọt (Bubble Sort) tiếp nhận danh sách này và thực hiện các phép hoán vị (Swap) để phân loại, qua đó trích xuất ra tối đa 3 phương tiện có độ tương đồng cao nhất đẩy lên đầu danh sách trả về.

V. TÀI LIỆU THAM KHẢO

- [1]. K. Thulasiraman, M. N. S. Swamy, Graphs: Theory and Algorithms, John Wiley & Sons, 1992.
- [2]. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, Introduction to Algorithms (Fourth Edition), The MIT Press, 2022.
- [3]. Jahn timer Kalluru, Enhancing Data Accuracy and Efficiency: An Overview of Fuzzy Matching Techniques, International Journal of Science and Research (IJSR), 2023.
- [4]. G. Navarro, A Guided Tour to Approximate String Matching, ACM Computing Surveys, 2001.
- [5]. A. Leshchenko, Overview and Practical Implementation of Fuzzy Search Algorithms, Transaction of Scientific Papers of the Novosibirsk State Technical University, 2019.
- [6]. Neha Makariye, Towards Shortest Path Computation Using Dijkstra Algorithm, 2017 International Conference on IoT and Application (ICIOT), 2017.
- [7]. Kalaphath Kounlaxay, Implementation of Shortest Path Finding using Dijkstra's Algorithm, Souphanouvong University Journal Multidisciplinary Research and Development, 2024.
- [8]. M. R. Wayahdi, Subhan Hafiz Nanda Ginting, Dinur Syahputra, Greedy, AStar, and Dijkstra's Algorithms in Finding Shortest Path, International Journal of Advances in Data and Information Systems, 2021.